

The Mental Model.

A living, daily-updated map of every concept learned — built to be skimmed in five minutes and remembered for good.



Java + Spring Boot



Google Cloud Bigtable



GKE Deployment

DAY 1 OF 17

DEADLINE: SEP 25, 2026



Ankit · 9 yrs Data Engineering

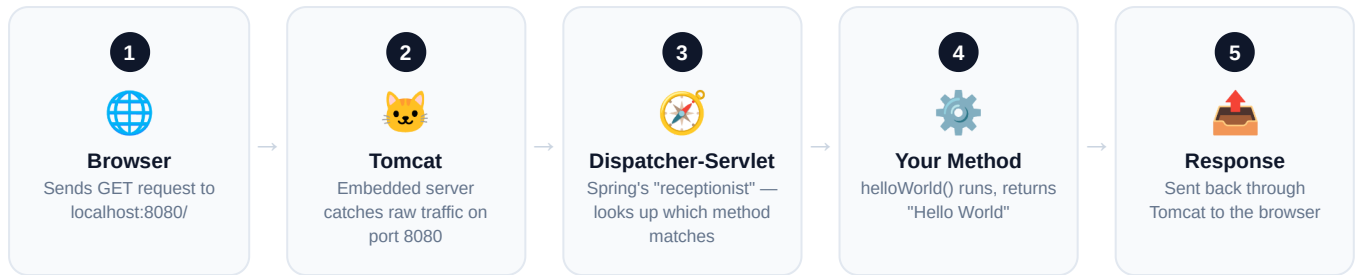
Spark · Python · SQL · BigQuery · Composer → now learning Java from zero

OUT-OF-DOMAIN SKILLS TEST

Pivoted Sept 8, 2026

How one HTTP request travels through Spring Boot

The picture to keep in your head every time you see `@RestController` or `@GetMapping`.



Spring builds its lookup table (step 3) once, at startup — by scanning every class tagged `@RestController` and every method tagged `@GetMapping`. This is **inversion of control**: the framework decides when your code runs, not the other way round.

```
package com.ankit.bigtable_api;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController // "this class holds web routes — scan me"
public class HelloController {
    @GetMapping("/") // "run me for GET /"
    public String helloWorld() {
        return "Hello World";
    }
}
```

`@RestController`

Marks a whole class as "contains web endpoints." Combines the job of FastAPI's `app = FastAPI()` + auto-JSON serialization.

`@GetMapping("/")`

Marks one method as the handler for GET requests to a path. Direct equivalent of `@app.get("/")`.

Annotation (`@...`)

A "sticky note" on a class/method that tells Spring what role it plays. Does nothing by itself — Spring reads it at startup.

`package`

A namespace that must match the file's real folder path exactly. Java enforces this; Python treats it as convention only.

static vs. new — when do you need an object?

The question that unlocked why `SpringApplication.run()` is called directly on the class name.

⚡ static — the light switch

Belongs to the **class itself**. No object needed — call it directly: `ClassName.method()`. Use when the action doesn't need to remember anything specific to one instance. `main()` is static because when the JVM starts, nothing exists yet to call it "on."

🔧 instance — the assembled lamp

Needs an object built first: `new ClassName()`, then call methods on *that* object. Use when each object must hold its own distinct data — e.g. two `Bigtable` rows with different data need two separate objects.

Python → Java translation table

Same concepts, stricter syntax. Refer back here whenever new Java code feels unfamiliar.

CONCEPT	PYTHON / FASTAPI	JAVA / SPRING BOOT
Install dependencies	<code>pip install fastapi</code>	<code>pom.xml</code> + Maven
App entry point	<code>main.py</code>	<code>XApplication.java</code> + <code>main()</code>
Run the server	<code>uvicorn main:app</code>	<code>mvnw spring-boot:run</code>
Define a route	<code>@app.get("/")</code>	<code>@GetMapping("/")</code>
Mark a class as routes	(implicit via app object)	<code>@RestController</code>
Config / secrets	<code>.env</code> file	<code>application.properties</code>
Dependency manifest	<code>requirements.txt</code>	<code>pom.xml</code>
Compiled cache (ignore in git)	<code>__pycache__</code> /	<code>target/</code>

Today's Windows gotchas — filed for next time

- ✓ Zip files can double-nest on extraction — always verify with `Test-Path` before trusting a path.
- ✓ New PATH entries need a **full VS Code restart**, not just a new terminal tab.
- ✓ `git add .` only reaches the current folder and below — run `git status` from the repo root to see everything.
- ✓ "Terminate batch job (Y/N)?" on Ctrl+C is a Windows `.cmd` quirk, not a Java/Spring behavior — just answer Y.

The daily Git ritual — non-negotiable

Every single day, in this order, no exceptions.

```
1  git status → confirm a clean tree before starting
2  <do the day's work> → code, run, verify it works
3  git add . → from the repo root
4  git commit -m "Day X (Java): ..." → descriptive, never "update"
5  git push → checkpoint saved, safe to stop
```

Day 1 — what actually got built

- ✓ JDK confirmed (Java 22) and Maven installed + added to PATH manually.
- ✓ Spring Boot project scaffolded via `start.spring.io`, living in `java-api/` inside the existing repo.
- ✓ Diagnosed and fixed a missing `spring-boot-starter-web` dependency by reading `pom.xml` directly.
- ✓ Confirmed embedded Tomcat starts on port 8080.
- ✓ Wrote and ran the first real endpoint — `GET /` → "Hello World" — via a separate Controller class.
- ✓ Traced a full request lifecycle end-to-end: browser → Tomcat → `DispatcherServlet` → method → response.

NEXT UP — DAY 2

Java classes & objects in depth, plus path and query parameters in Spring (`@PathVariable` / `@RequestParam`) — the direct equivalent of `GET /items/{item_id}` from the original Python track.